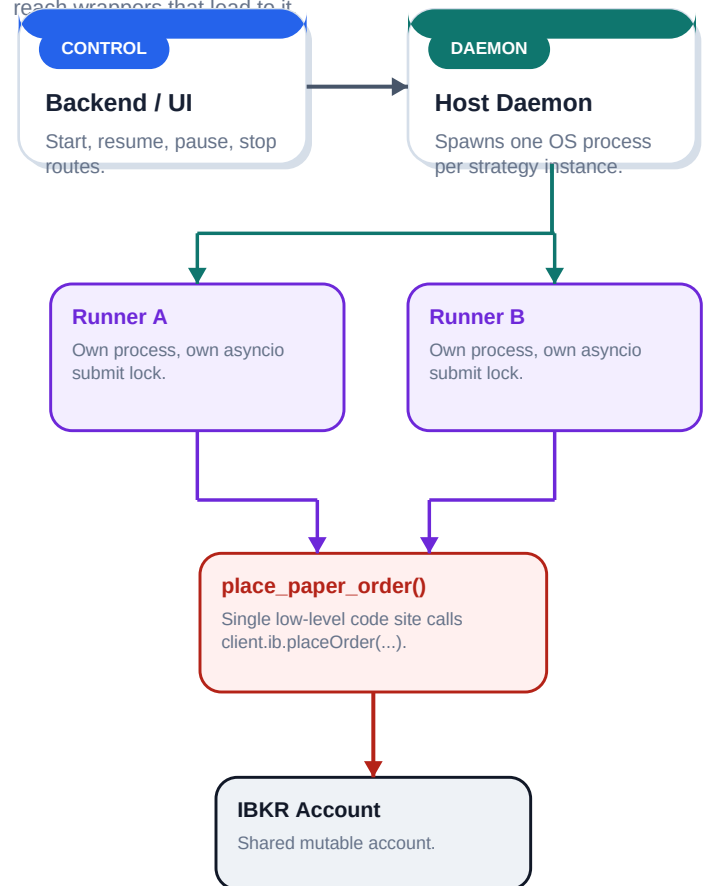


1. Big Picture: Current R2 vs Future R3

The current code has good local safety gates, but multiple runner processes can still reach the shared broker writer. The future design moves broker writes behind one account owner.

Current code today: R2 multi-process writers

One low-level placeOrder call site, but more than one live runner process can reach wrappers that lead to it.



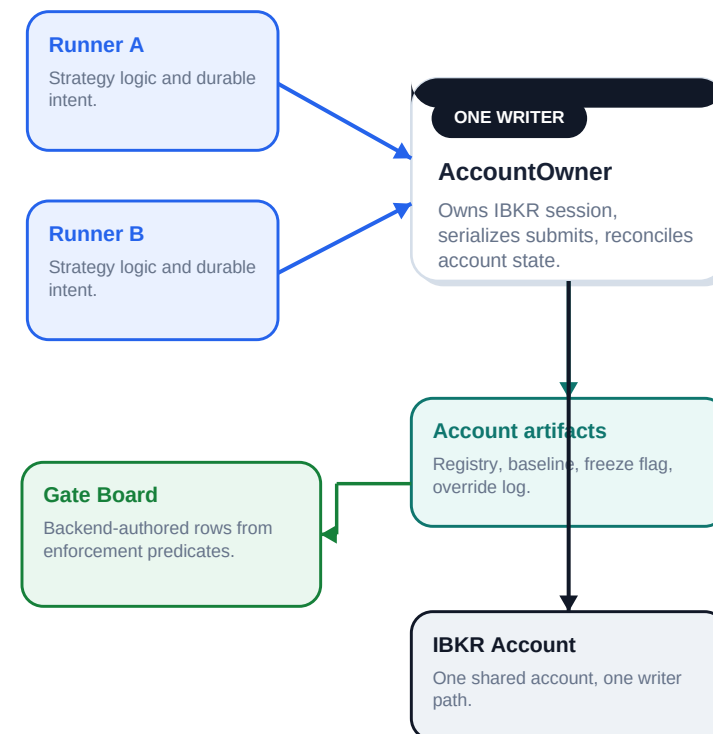
implemented gates

partial account scope

gap: no freeze

Future design: R3 AccountOwner

Runners keep doing strategy work, but they only emit durable intents. AccountOwner is the only process allowed to place broker orders.



preflight

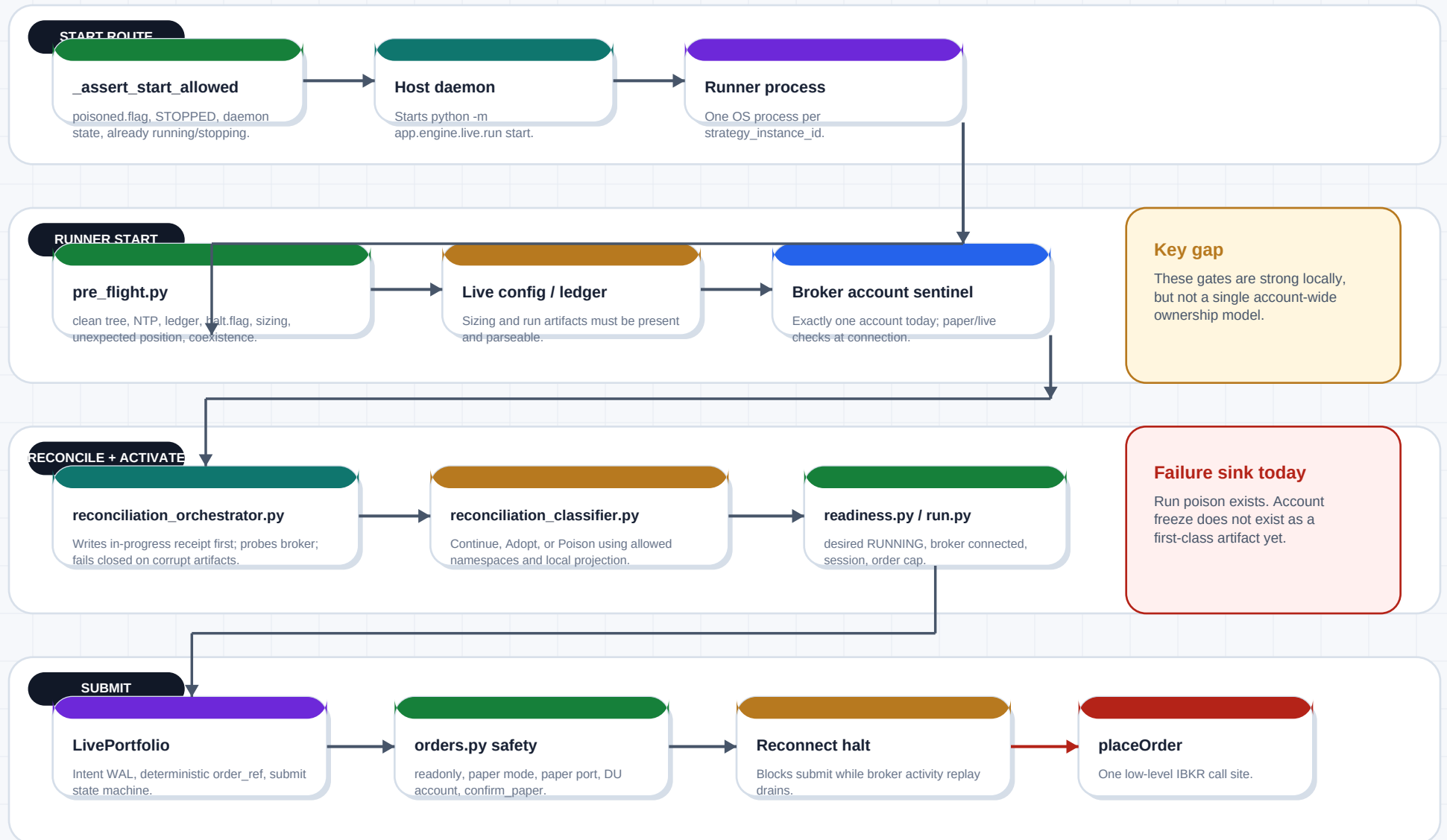
reconcile

submit

recovery

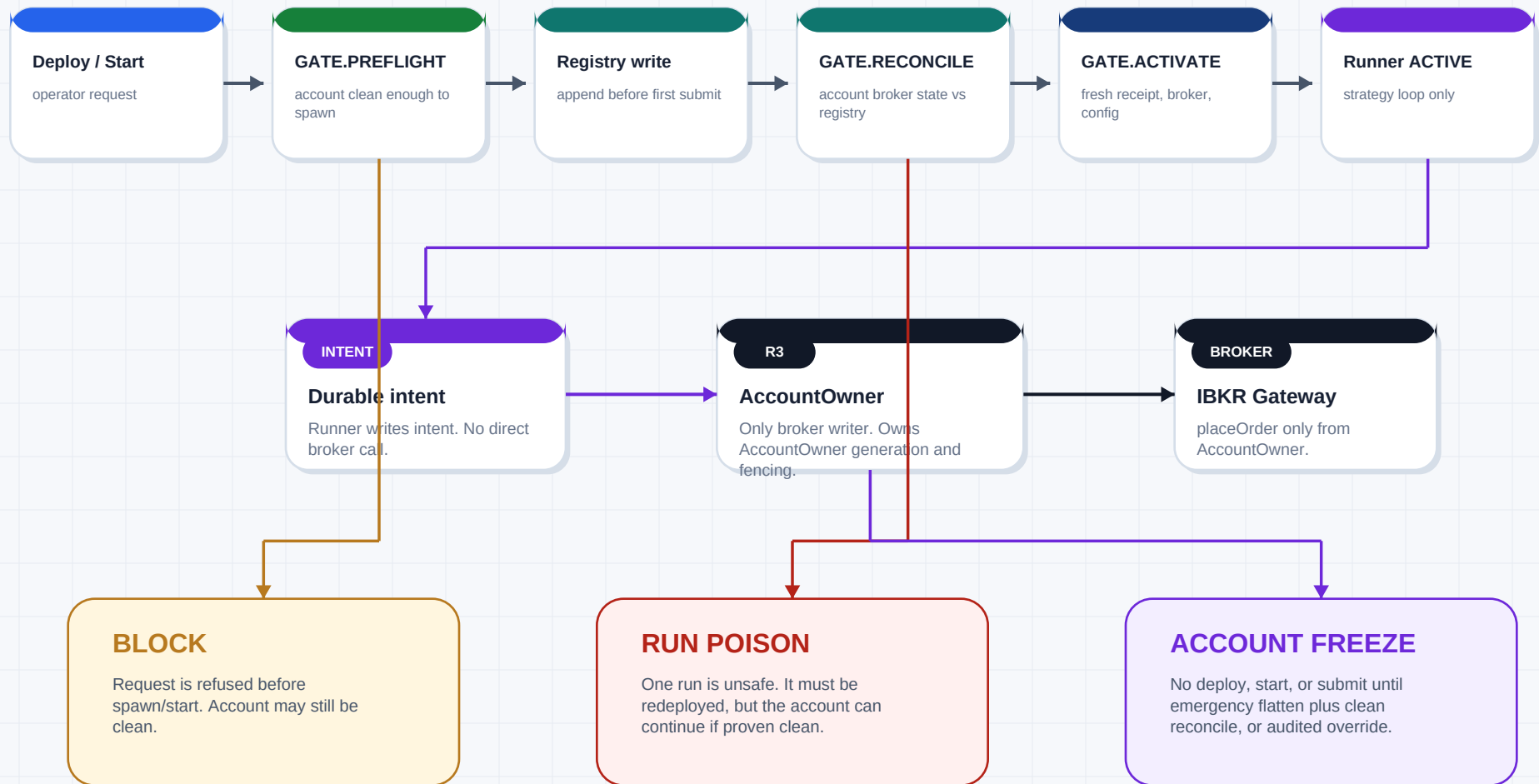
2. Current Code Path: Where Gates Exist Today

This is intentionally a map of the code as it is now. The safety is real, but most checks are run-scoped or process-local, not account-wide.



3. Future Lifecycle: Account-Scoped Gates

The future flow puts account gates before spawning and keeps account recovery separate from run poison. Dense pieces are split into later pages.



Rule of thumb: poison is local to one run; freeze is account-wide and stops the whole shared broker account.

4. Gate Board Contract

One row equals one enforcement predicate. The cockpit displays backend-authored safety decisions instead of inferring meaning from raw state.

Backend-authored gate board

Each row is a visible version of the enforcement result. If the mutation would block, the row must not say pass.

gate_id	phase	scope	status	source + enforcement	operator next step
start.poison_sentinel	start	run	BLOCK	_assert_start_allowed -> Start route	redeploy run
account.unresolved_exposure	preflight	account	FREEZE	unresolved_exposure.flag -> preflight	flatten + reconcile
reconcile.account_classifier	reconcile	account	PASS/FREEZE	AccountOwner classifier -> activate	adopt or recover
submit.paper_safety	submit	order	BLOCK	_enforce_paper_safety -> orders.py	fix broker mode
submit.uncertain_ack	submit	order	UNKNOWN	submit resolver -> AccountOwner	probe broker
watchdog.proof_before_exit	recovery	process	FREEZE/PASS	watchdog executor -> shutdown	prove or freeze

Field set
gate_id, phase, scope, status,
source_of_truth, enforcement_point,
blocks, next_step, evidence.

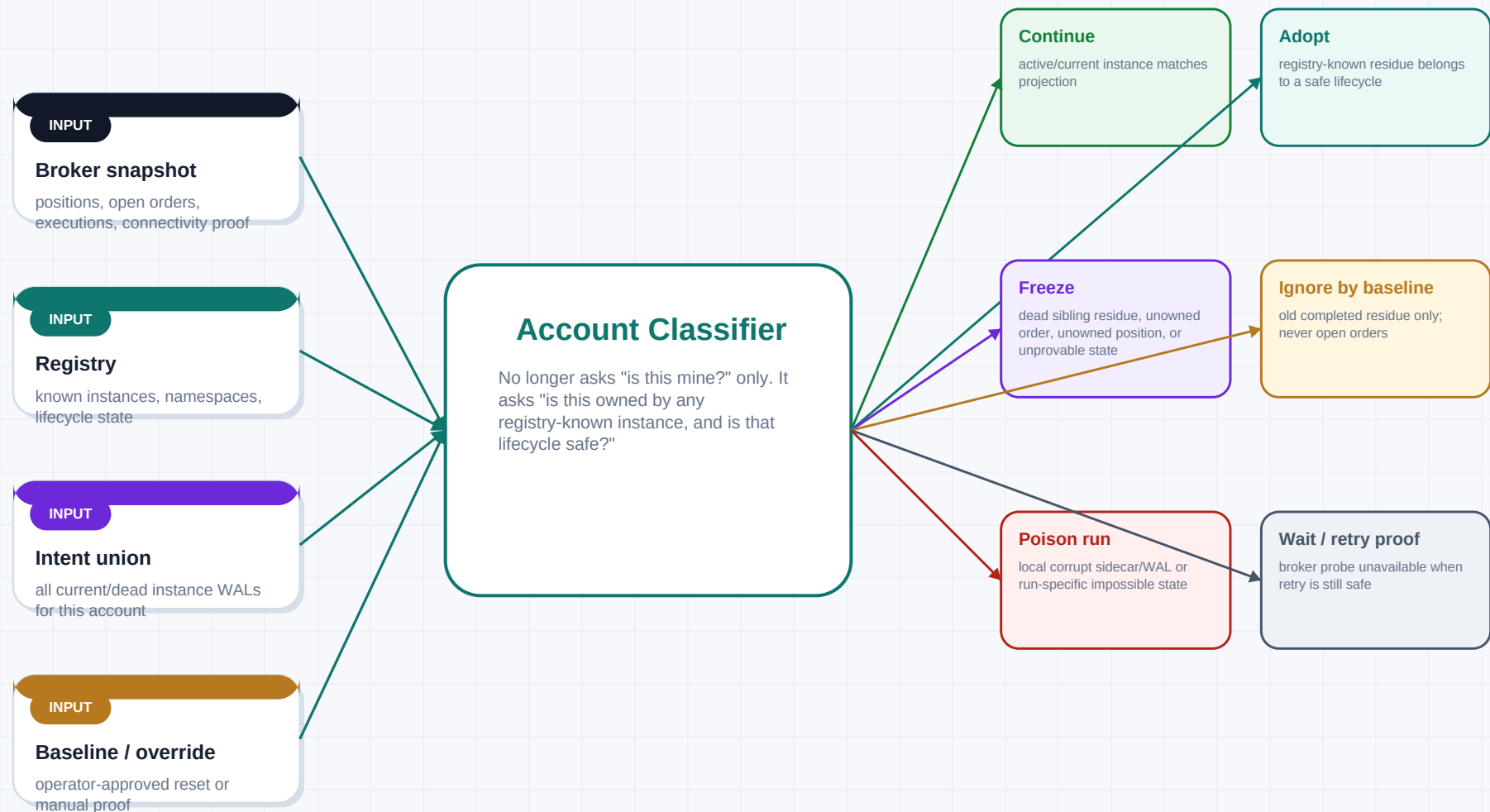
Hard rule
No parallel UI projection. Display the
enforcement result itself.

Angular role
Render table/timeline. Do not decide
what a gate means.

Operator value
The next step is visible: reconnect,
flatten, redeploy, override, or wait.

5. Sub-Chart: Account-Scoped Classification

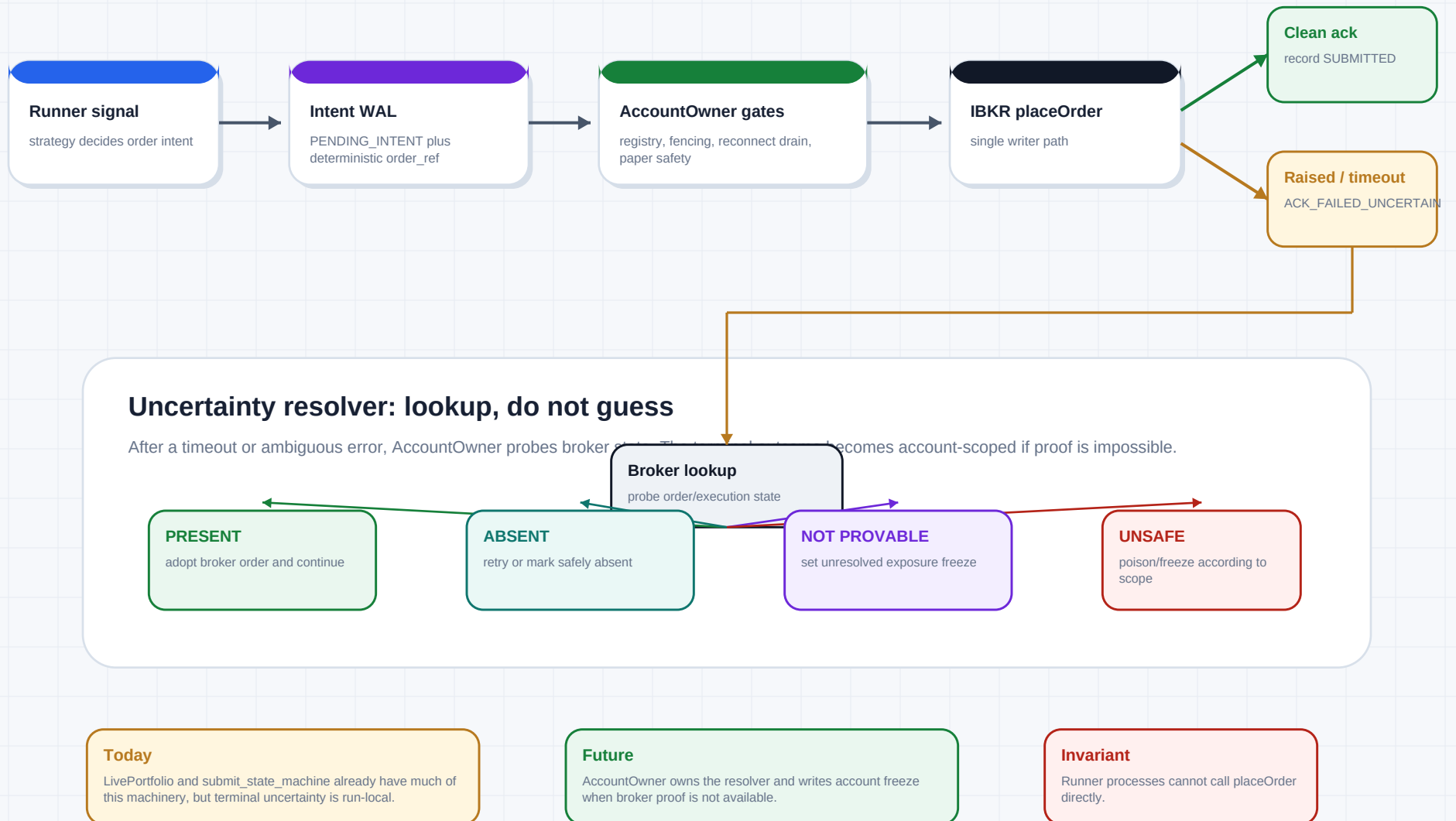
The classifier should answer for the whole account. It compares broker artifacts against registry-known instances, intent history, baseline, and override evidence.



Important: unowned open orders and unowned live positions are not baselineable; they route to account freeze/recovery.

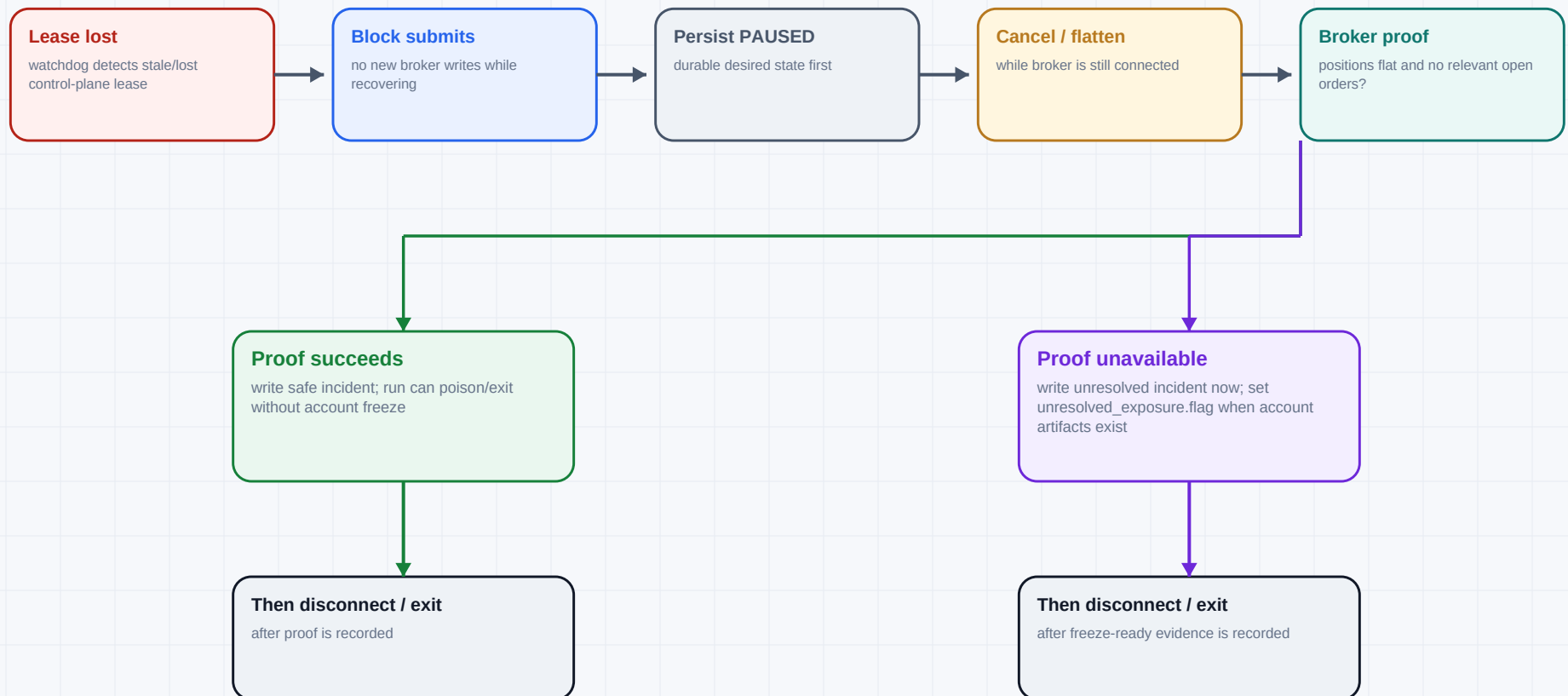
6. Submit and Uncertainty Resolution

Zoom into PROC.SUBMIT. Timeout means unknown, not failure; the future R3 path routes every broker write through AccountOwner.



7. Sub-Chart: Watchdog Recovery - Disconnect Last

This is the urgent live-bug fix. If lease/control is lost, we preserve proof or unresolved evidence before disconnecting the broker.



Clear path: emergency flatten/cancel -> broker probe -> account classifier clean -> append unfreeze event.

Override path: if IBKR is unreachable, operator documents manual flat/no-open-orders proof; next reconnect must reconcile before submit.

8. Implementation Roadmap: Build the Interlocks in Order

The order matters. Ship the proven live-bug fix before large account ownership migration, then move toward AccountOwner as the single writer.

